

Predicción de Abandono de Clientes (Churn)

AndesLink Servicios Digitales S.A.

Alumnos: Facundo Zubeldia · Gonzalo Martín González Nastovich · Agustín Meza

Profesor: Diego Mosquera

Fecha: Mayo 2026

1. Problema de Negocio

AndesLink Servicios Digitales S.A. comercializa planes de suscripción mensual para servicios digitales orientados a consumidores finales. Durante los últimos trimestres, la empresa detectó una tasa creciente de cancelación voluntaria de clientes (churn), lo que impacta negativamente en los ingresos recurrentes, el costo de adquisición de nuevos usuarios y la estabilidad del flujo de caja.

El objetivo analítico es entrenar un modelo de clasificación binaria capaz de estimar la probabilidad de que un cliente abandone el servicio, a partir de variables de comportamiento, antigüedad, facturación y relación con el servicio.

Variable objetivo: churn — valor 1 si el cliente abandonó el servicio, 0 si continúa activo.

2. Dataset

Procedencia: Dataset sintético diseñado para simular el comportamiento real de clientes de una empresa de suscripciones digitales.

Tamaño: 5.000 registros · 16 columnas (15 features + 1 variable objetivo).

Versionado: El dataset está versionado con DVC. El archivo CSV no se commitea al repositorio; se almacena el archivo churn_sintetico.csv.dvc con el hash del dato. El archivo real se descarga con dvc pull.

Variables principales:

Variable	Tipo	Descripción
----------	------	-------------

tenure_months	Numerica	Meses de antigüedad del cliente
monthly_charge	Numerica	Cargo mensual
total_charges	Numerica	Cargo total acumulado (derivada)
support_tickets	Numerica	Tickets de soporte generados
late_payments	Numerica	Pagos tardíos registrados
avg_monthly_usage_gb	Numerica	Uso promedio mensual en GB
contract_type	Categorica	Tipo de contrato: mensual / anual / bianual
payment_method	Categorica	Metodo de pago
internet_service	Categorica	Tipo de servicio: fibra / cable / movil / ninguno
region	Categorica	Region geografica del cliente
has_streaming	Binaria	Tiene servicio de streaming (0/1)
has_security_pack	Binaria	Tiene paquete de seguridad (0/1)
num_products	Numerica	Cantidad de productos contratados
customer_age	Numerica	Edad del cliente
is_promo	Binaria	Ingreso con promocion (0/1)

Limitaciones del dataset:

- Dataset sintético: los patrones pueden no reflejar el comportamiento real de clientes.
- total_charges es una variable derivada (tenure x monthly_charge), introduce colinealidad ($r=0.87$ con tenure_months).
- Las correlaciones con churn son bajas (max $r=-0.17$), sugiriendo dependencias no lineales.

3. Analisis Exploratorio (EDA)

3.1 Distribucion de la variable objetivo

El dataset presenta un desbalance moderado: 66% de clientes que se quedan (clase 0) vs. 34% que se van (clase 1). Este desbalance determina que accuracy no sea la metrica correcta: un modelo que prediga siempre clase 0 tendria 66% de accuracy sin detectar ningun cliente en riesgo.

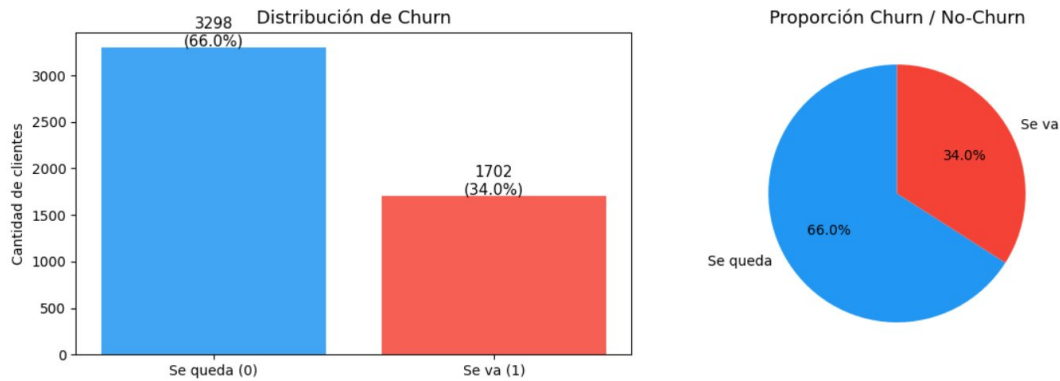


Figura 1: Distribucion de la variable churn en el dataset.

3.2 Variables numericas vs. churn

Los boxplots muestran diferencias clave: tenure_months (Med 0: 39 vs Med 1: 29) y support_tickets (Med 0: 1 vs Med 1: 2) son las variables con mayor separacion entre grupos, indicando que clientes nuevos con muchos tickets tienen mayor propension al churn.

Variables numéricas: clientes que se quedan vs. que se van

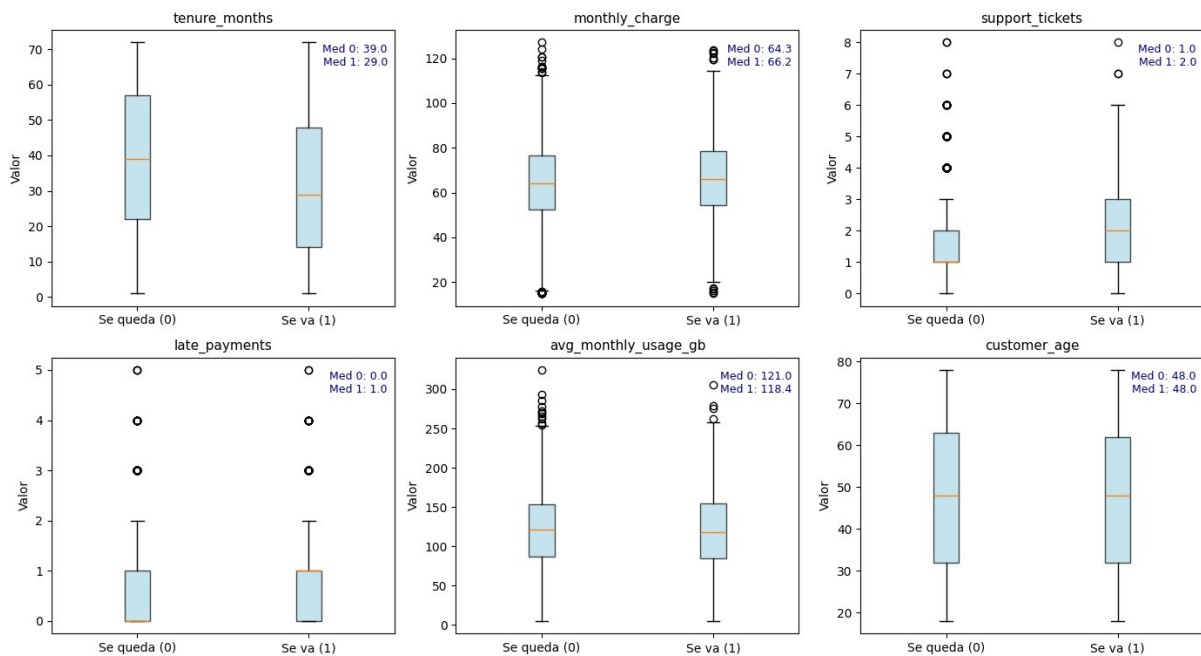


Figura 2: Distribucion de variables numericas — clientes que se quedan vs. que se van.

3.3 Matriz de correlacion

Se confirma la colinealidad entre total_charges, tenure_months ($r=0.87$) y monthly_charge ($r=0.41$). La variable con mayor correlacion con churn es tenure_months ($r=-0.17$): mayor antigüedad implica menor riesgo de abandono.

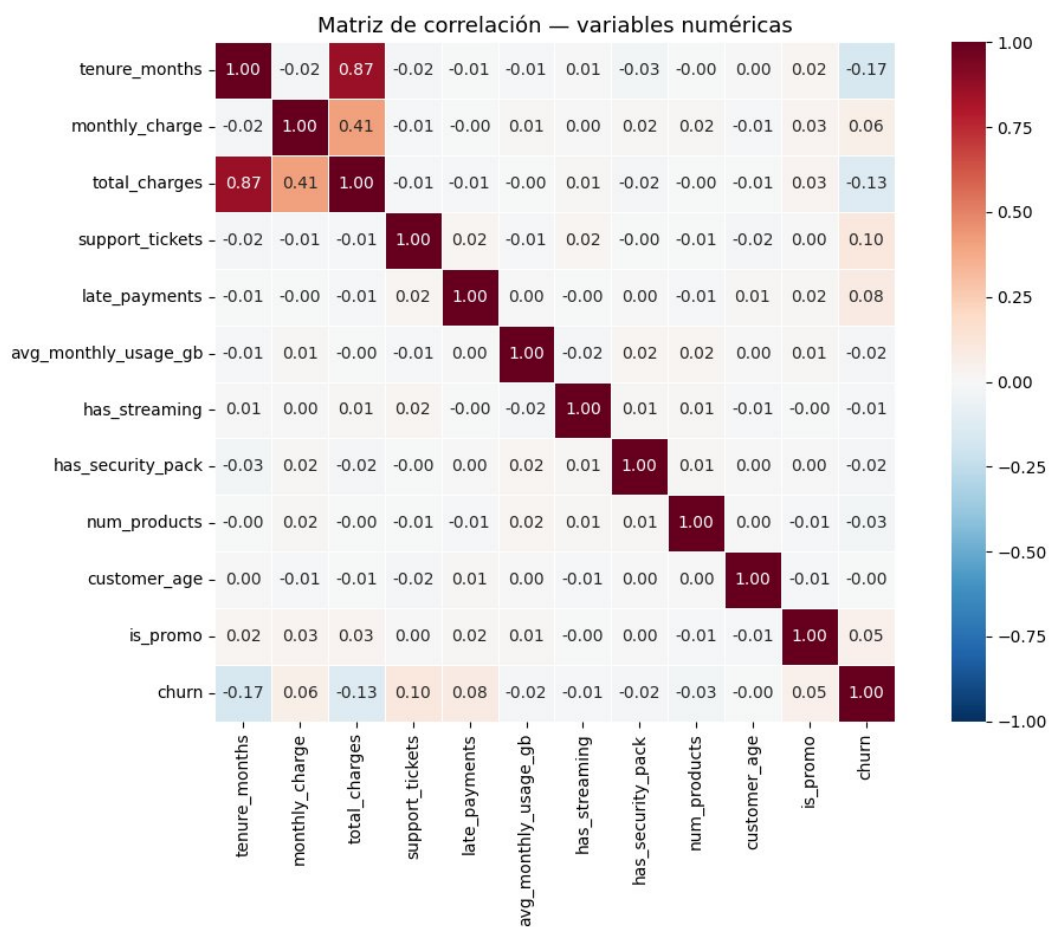


Figura 3: Matriz de correlacion entre variables numericas.

3.4 Variables categoricas vs. churn

El analisis de tasa de churn por categoria revela hallazgos de negocio relevantes:

- contract_type: contratos mensuales tienen 47.5% de churn vs. 11.6% en bianuales.
- internet_service: servicio movil tiene 50.9% de churn; fibra solo 26.8%.
- payment_method: efectivo tiene mayor churn (40.5%), asociado a menor automatizacion.
- region: sin diferencias significativas (33-34%); el churn no tiene componente geografico.

Tasa de churn por variable categórica

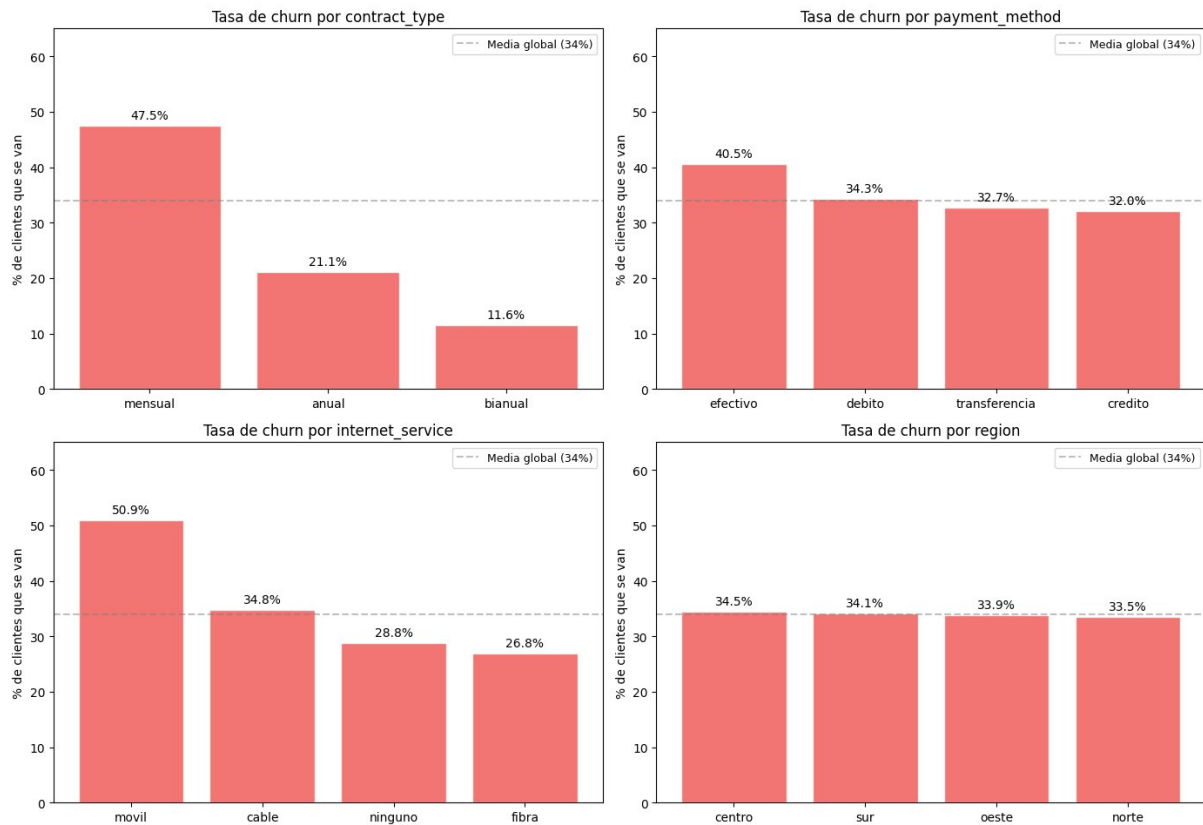


Figura 4: Tasa de churn por variable categorica.

4. Preparacion de Datos

Limpieza:

- Eliminacion de customer_id (identificador unico, no predictivo).
- Verificacion y eliminacion de duplicados.
- Sin valores nulos detectados en el dataset.

Separacion train/test:

Division 80/20 con random_state=42. El 20% (1.000 registros) se reservo para evaluacion final sin participar en el entrenamiento.

Pipeline de preprocesamiento:

Se implemento un Pipeline de scikit-learn que aplica transformaciones de forma consistente:

- Variables numericas: StandardScaler (media 0, desvio 1). Necesario para convergencia de Logistic Regression.

- Variables categoricas: OneHotEncoder con handle_unknown='ignore'. Robusto ante categorias nuevas en inferencia.

El pipeline completo (preprocesador + clasificador) se serializa en un unico artefacto .pkl, garantizando que la inferencia aplique exactamente las mismas transformaciones.

5. Entrenamiento y Evaluacion

5.1 Eleccion de metricas

Con 34% de churn, la accuracy no es la metrica correcta. La metrica principal es el F1-score de la clase 1 (churn), que balancea precision y recall. El Recall es especialmente relevante: mide cuantos de los clientes que realmente se van son detectados. Un recall bajo implica campanas de retencion ineficaces. El ROC-AUC se usa para comparar la capacidad discriminativa de los modelos independientemente del umbral.

5.2 Comparacion de modelos

Se compararon tres algoritmos, todos con `class_weight='balanced'` para penalizar errores sobre la clase minoritaria:

Modelo	Accuracy	F1 (churn)	Recall (churn)	ROC-AUC
Logistic Regression	0.703	0.609	0.704	0.768
Decision Tree	0.671	0.582	0.698	0.729
Random Forest	0.712	0.444	0.351	0.732

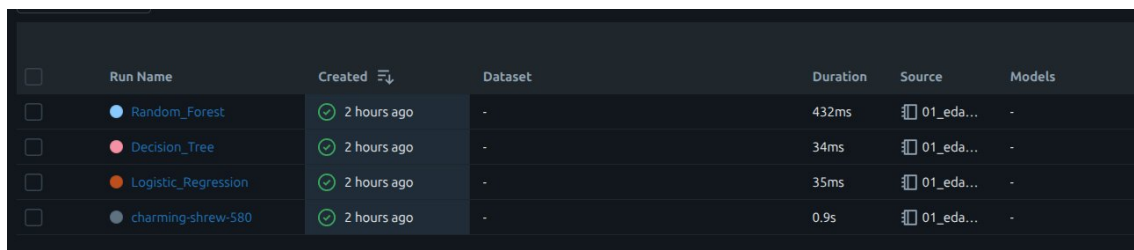
Tabla 1: Comparacion de modelos. Logistic Regression = modelo seleccionado.

5.3 Modelo seleccionado: Logistic Regression

Seleccionada por tener el mayor F1-score (0.609), Recall (0.704) y ROC-AUC (0.768). Random Forest, a pesar de tener el mayor accuracy (0.712), tuvo el peor F1 y Recall (0.444 y 0.351). Esto confirma por que accuracy no es la metrica correcta: RF aprendio a predecir bien la clase mayoritaria pero fallo en detectar clientes en riesgo de fuga.

6. Tracking de Experimentos con MLflow

Todos los experimentos fueron registrados con MLflow bajo el experimento 'AndesLink_Churn_Prediction'. Se loguearon F1, Recall y ROC-AUC por cada modelo. La base de datos se almacena en notebooks/mlflow.db.



☐	Run Name	Created	Dataset	Duration	Source	Models
☐	Random_Forest	2 hours ago	-	432ms	01_eda...	-
☐	Decision_Tree	2 hours ago	-	34ms	01_eda...	-
☐	Logistic_Regression	2 hours ago	-	35ms	01_eda...	-
☐	charming-shrew-580	2 hours ago	-	0.9s	01_eda...	-

Figura 5: UI de MLflow con los 3 runs del experimento.

Estrategia de reproducibilidad:

- Git: versionado del codigo fuente y archivos de configuracion.
- DVC: versionado del dataset. El CSV no se commitea al repo (churn_sintetico.csv.dvc).
- MLflow: tracking de experimentos, hiperparametros y metricas.
- environment.yml + requirements.txt: entorno reproducible con versiones fijas.

7. Conclusiones y Limitaciones

Se desarrollo un pipeline MLOps completo para la Entrega 1: desde la exploracion del dataset hasta el modelo serializado listo para inferencia. La Regresion Logistica con `class_weight='balanced'` es el modelo final, con $F1=0.609$ y $Recall=0.704$ en la clase churn.

Limitaciones:

- Dataset sintetico: puede no reflejar comportamiento real de clientes.
- Sin optimizacion de hiperparametros (GridSearch/RandomSearch). Hay margen de mejora.
- `total_charges` introduce colinealidad; podria evaluarse su eliminacion.
- Recall del 70.4%: el modelo aun no detecta el 30% de los clientes en riesgo.

Proximos pasos (Entrega 2):

- Exponer el modelo como API REST con FastAPI.
- Construir dashboard con Streamlit para usuarios no tecnicos.
- Contenedorizar la solucion con Docker y Docker Compose.